CRAY
THE SUPERCOMPUTER COMPANY

# *Using the Cray Comparative Debugger (CCDB)*

Andrew Gontarek

PE Debugger Group
Cray Inc.

S-0036-10
Release 1.2
June 2012

©2007-2012 Cray Inc. All Rights Reserved
Copyright (c) 1996-2012 by Guardsoft Pty Ltd.
GUARD is a trademark of Guardsoft Pty Ltd.

## Contents


Chapter 1 – Quick Start ..... 3
Installing ccdb ..... 3
Loading ccdb modules ..... 3
Running ccdb ..... 3
Chapter 2 - Introduction ..... 3
Chapter 3 – Definitions & Syntax ..... 4
Chapter 4 - Usage ..... 5
Application Invocation ..... 5
Comparisons and Assertions ..... 6
Chapter 5 – Debugger Commands ..... 6
Interactive Mode Commands ..... 7
Info Subcommands ..... 17
Set Subcommands ..... 18
Show Subcommands ..... 22
Usage Subcommands ..... 24
Blockmap Deferred Mode Commands ..... 25
Graph Deferred Mode Commands ..... 27

# Chapter 1 – Quick Start

This chapter describes how to start using ccdb in the minimal amount of time.

## Installing ccdb

This tool is distributed as a PE package and should be installed by default.

## Loading ccdb modules

From the command line, type:

```
$ module load ccdb
```

This loads the default module for ccdb that is installed on the system.

## Running ccdb

After the ccdb module is loaded, allocate the compute node resources required for the debugging session. For example, if your site uses PBS, type:

```
$ qsub –I –l mppwidth=W
```
(where W is a scalar value)

This command requests compute nodes from the system and starts an interactive session with access to the requested compute nodes.

Next, issue the `ccdb` command to start the comparative debugger:

```
$ ccdb
ccdb - Cray Comparative Debugger
Copyright (c) 2007-2012 Cray Inc. All Rights Reserved.
Copyright (c) 1996-2012 by Guardsoft Pty Ltd
dbg all>
```

At this point, you are ready to begin entering ccdb commands. For example, to invoke an application to debug, type:

```
dbg all> invoke $myapp{W} /path/to/app
```
(Where W is the number of application nodes)

As long as you have enough available idle compute nodes allocated, new invocations of an application can be issued using the invoke command. This will transparently launch the application onto the nodes using aprun. See chapter 4 of this manual for more information.

# Chapter 2 - Introduction

### ccdb is a relative debugger

Unlike other debuggers, ccdb allows application developers to compare data structures from many different applications as they are executing. This information can then be used as an aid in identifying the location of errors.

### ccdb is a conventional interactive parallel debugger

ccdb can be used to debug parallel and serial applications using interactive commands, such as setting breakpoints and displaying the program state. Because ccdb is a parallel debugger, it allows debugging commands to be applied to sets of parallel processes.

### ccdb is built on top of gdb

GDB is a versatile debugger that has widespread adoption, a large support community, and ongoing active development. GDB provides the low-level debug services with which ccdb interacts to enable the task of comparative debugging. The syntax used in ccdb has also been heavily influenced by the High Performance Debugging Forum's HPD standard.

## Chapter 3 – Definitions & Syntax

The command set used in ccdb is derived from gdb. Users will find that the syntax and feel of the debugger is similar to that of gdb, the most notable difference being support for parallel applications which gdb currently lacks.

For example, a typical command is formatted as:

```
command [arg1]…[argN]
```

Arguments to commands can consist of keywords or expressions.

Expressions are comprised of the usual C-like operators, constants, internal variables, and program variables commonly used inside of gdb. Enclosing an expression with quotes denotes that this portion of the expression should be passed directly to the gdb backend and not interpreted by any built-in ccdb specific functions.

ccdb supports convenience variables for storing any temporary variables to be used in a later expression. These variables are denoted by a dollar sign followed by a name; for example `$var`.

When parallel applications are debugged using ccdb, the application ranks are arranged into groups called *process sets*. The default process set is named *all* and contains the ranks of every application that is currently being debugged. Users can refer to specific application ranks by defining a new process set containing the desired ranks and *focusing* on that named set. A user can also refer to every rank of a specific application by focusing on the named process set that was specified during the original invoke of the application. When a user focuses on a process set, it becomes the *current set*. Commands are only applied to ranks contained in the current set. To reiterate, by default the process set *all* is the *current set* and all commands issued inside ccdb will act upon every rank of the debugged applications.

In ccdb, variables of a target application are referenced using a syntax consisting of an optional process set argument and an identifier or gdb expression, which is separated by `::`. If no process set is declared, the command is sent to all ranks in the current set. For example, to refer to the variable `var` in the process set `$s`, use the argument `$s::var`. To use more complex expressions that are not natively supported by ccdb, you can enclose the expression with quotes; for example, to call the `sizeof` function on the variable `var` for the process set `$s` use `$s::"sizeof(var)"`. The use of quotes denotes that the expression contains something that should be passed directly to gdb.

If a convenience variable is an array, specific elements can be referenced using ccdb array syntax which uses curly braces, such as `$s{1}::var`. When referencing program variables that are arrays, you use the language specific format. For example, to print elements of the c variable `var` in process 1 of procset `$s` use `$s{1}::var[2][3]`. To print elements of the Fortran variable `var` use `$s{1}::var(3,2)`. For program variables, ccdb also supports the notion of array slices. An array can be sliced by specifying an array index as a range, such as `var[3..10][4]`. This will operate on the specified subset of the entire array.

## Chapter 4 - Usage

ccdb is started by issuing the `ccdb` command in a terminal. Once the client is started, you will see an introductory message, followed by a command prompt:

```
$ ccdb
ccdb - Cray Comparative Debugger
Copyright (c) 2007-2012 Cray Inc. All Rights Reserved.
Copyright (c) 1996-2012 by Guardsoft Pty Ltd
dbg all>
```

At this point, you can begin issuing commands to the debugger. To view a list of all valid commands, type `help`. Help text containing valid command syntax is displayed.

### Application Invocation

New applications to debug are launched using the invoke command. Invoke specifies the name of the executable and the number of processes to create.

The syntax of the invoke command is:

```
invoke <process set expression> <application> [--args <arguments>]
       [--aprun-args <aprun_arguments>] [--aprun-input <path_to_file>]
```

The process set expression argument is a list of one or more convenience variables that are used as the process set name(s) for the application. These convenience variables can either be a scalar, such as `$app1`, or arrays of a given length, such as `$app2{8}`. If more than one convenience variable is provided, each should be separated using a comma, such as `$app1,$app2{8}`. The summation of all convenience variables, including the length of any array, is used to determine the number of application ranks that will be launched.

For example, if you provide the following process set expression:

```
$app1,$app2{8},$app3{3}
```

You will have 12 ranks in your application. This is calculated by summing:

`$app1` = 1 rank
`$app2{8}` = 8 ranks
`$app3{3}` = 3 ranks
**Total:** 12 processes

The total number of ranks is used transparently to create the aprun command and launch the application. The ranks of the application are assigned in ascending order to each declared convenience variable, in a left to right fashion.

The `args` option is used to pass arguments directly to your application. The `aprun-args` option is used to pass aprun arguments directly to the aprun command used when launching the application. The `aprun-input` option is used to pass the location of an input file that aprun will redirect its stdin to for an application that requires addition setup from stdin. All of the arguments passed to the options are a list of arguments that are enclosed by strings.

For example, if you wish to provide `–N1 –L134` to the aprun used to invoke your application, you would use:

```
--aprun-args “-N1 –L134”
```

## Comparisons and Assertions

Comparisons and assertions are commands that enable the task of comparative debugging. The `compare` command is used to instantaneously compare program data relative to the set of invoked applications. For example, consider two single rank applications that have been invoked with the convenience variable names `$app1` and `$app2`. A breakpoint is set in both and the applications are instructed to continue until the breakpoint is reached. At this breakpoint, there is a variable named `var` in the current execution scope of the two applications that you want to compare. Do so by issuing the following command:

```
compare $app1::var = $app2::var
```

If the display mode is set to the default of "text" mode, the differences between these two variables are displayed immediately.

The `assertion` command is employed when you want to define an equality condition to check for during application execution. In order to define assertions before application execution, use the `graph` command. Assertions are encapsulated inside of a graph, and defined using a variable reference similar to the syntax used in the `compare` command at a particular locale inside of the application source. The general syntax of an assertion looks like the following:

```
graph $g
      assert $app1::var@"app1.f":10 = $app2::var@"app2.f":20
end
```

This command sequence tells the debugger to create a graph called `$g` containing a single assertion command. This assertion compares the value of `var` from `$app1` at line `10` in the source file `"app1.f"` to the value of `var` from `$app2` at line `20` in the source file `"app2.f"`. Note that graphs can contain as many assertions as you want, but each additional assertion defined increases the time required to complete a full iteration of the graph. You can then issue the `start $g` command to instruct the debugger to begin execution of the graph. Note that before an assertion can be defined or a graph run, applications that correspond to the assertion definitions in the graph must have been invoked previously and be at their initial startup barrier. When a graph is started, breakpoints are automatically inserted at the defined locale locations in the applications and both are continued. When a defined breakpoint is reached, the debugger extracts the contents of the defined variables in an architecture-neutral format and performs the comparison.

# Chapter 5 – Debugger Commands

The following list describes the types of commands available in ccdb.

Commands are classified into three target scope:

- **Application independent**: The command does not interact with an application.
- **Application targeted**: The command interacts with an application.
- **Application optional**: The command may or may not interact with an application, depending on its arguments.

These commands are used in coordination with two debugger modes:

- **Interactive mode**: Commands encountered are executed as soon as they are issued, generally resulting in client side actions being taken or communication with debug servers taking place.

- **Deferred mode**: Commands encountered are not executed, but rather saved into an internal buffer and executed at a later time after leaving the deferred mode context. In order to leave the deferred mode context, the user should issue the `end` command. The only way to enter this mode is to make use of a *deferred mode* command.

Commands such as `set` and `info` are interactive mode commands, whereas commands such as `graph` and `blockmap` will switch the debugger from *interactive mode* to *deferred mode*. The `end` command switches the debugger from *deferred mode* back to *interactive mode* and the block of commands entered is ready to be executed.

The commands defined in this chapter will take on the following form:

**Command name**

*(target scope)*

Summary: Brief summary of the command.
Usage: `command <arguments> [<optional argument>] [-<options>]`
Example command: `command example`
Shortcut: `valid shortcut name`

Sub-commands: If valid.
Arguments: Explanation of command arguments.

Description:
Detailed description of the command.

Each command begins with its bolded name and target scope (one of the three defined target scopes). This is followed by a brief summary of the command, its usage including any possible arguments it expects or options, a typical usage example of the command, followed by any valid shortcuts that can be used as shorthand for that command. Any sub-commands that are used in conjunction with this command are displayed, which is typical of deferred mode commands, as well as an explanation of the arguments defined in the commands usage. Finally a detailed description of the command is given.

## Interactive Mode Commands

### assign

*(application optional)*

Summary: Assignment command.
Usage: `assign <target> <expr> [-force]`
Example command: `assign j 20`

Arguments: `target` (target variable), `expr` (expression syntax), `-force` (force assignment)

Description:
Assign the value of an expression defined by `expr` to `target`. The target must evaluate to an l-value that is either a debugger variable (such as `$a`) or an application variable. If the target is the name of a debugger variable that does not exist, a new debugger variable with that name is automatically created. If the optional force option is specified, then the variable assignment is forced. This is used when the type of l-value defined by `expr` does not match the type of the l-value of `target`.

## backtrace

*(application targeted)*

Summary: Print stack frame backtrace command.
Usage: `backtrace [<num>]`
Example command: `backtrace`
Shortcut: `bt`

Arguments: `num` (pos/neg `num` frames)

Description:
Display a backtrace of the stack frames for all application ranks in the current process set. If `num` is supplied, the innermost `num` frames are displayed. If `num` is negative, the outermost `num` frames are displayed.

## break

*(application targeted)*

Summary: Set breakpoint command.
Usage: `break <location>`
Example command: `break main.c:20`
Shortcut: b

Arguments: `location` (line number, function name, or address)

Description:
Set a breakpoint in every process of the current process set. The location of the breakpoint can be defined as a line number, a function name, or "*" and an address. If an address is specified, it must begin with an asterix (*). For example, break *0x0000000000400693 would break at address 0x400693.

## blockmap

*(application independent)*

Summary: Blockmap creation command.
Usage: `blockmap <name>(<process>::<variable>)`
Example command: `blockmap decomposition(P::V)`

Deferred mode commands: `define`, `end`, `help`
Arguments: `name` (named variable), `process` (process set reference), `variable` (variable reference)

Description:
Begin creation of a new data decomposition scheme. Blockmap sub-commands are read until `end` is encountered. Blockmap is used to specify the reconstruction of distributed variables into the global representation of the data. Blockmap can be used in an assertion to compare data between instances of applications. When blockmap is used in an assertion, it is supplied with a process set and a local variable to operate on. It will take each ranks variable and recreate the global array based on the definitions provided to blockmap. Typically a blockmap is used inside of a dataflow graph in order to compare the global representation of the data space between two instances of applications that may vary in terms of scale but the global data size is the same.

To create a blockmap, the user must provide definitions for `data`, `distribute`, `grid`, and `dim_order`. The `data` definition is used to describe the size and dimensionality of the global reconstruction. The `distribute` definition is used to describe the distribution for each dimension of the reconstruction. The `grid` definition is used to describe the processing grid (or number of ranks contained in each dimension) for the reconstruction. The `dim_order` definition is used to describe the fastest varying dimension of the global reconstruction for rank assignment (i.e. which dimension contains sequential numbered ranks; and the order in which the other dimensions are incremented). See `usage define` for more information.

## compare

*(application optional)*

Summary: Compare command.
Usage: `compare <expr1> = <expr2>`
Example command: `compare i = j`

Arguments: `expr1` (variable expression), `expr2` (variable expression)

Description:
Compare (subtract) the value of expression `expr1` with the value of expression `expr2`. The values must be of the same size and shape, or a translation function (such as a blockmap) must be used. For complex data types such as arrays and structures, the comparison is performed on the zeroeth element only. The output of this command depends on the settings of `display`, `output`, and `error` variables.

## continue

*(application targeted)*

Summary: Continue debugging application.
Usage: `continue`
Example command: `continue`
Shortcut: c

Arguments: None.

Description:
Continue execution of every process in the current process set from its current location.

## create

*(application independent)*

Summary: Debugger variable creation command.
Usage: `create <varname>[[<dim>]]`
Example command: `create $new_var`

Arguments: `varname` (new named debugger variable), `dim` (optional array dimensions)

Description:
Create an internal debugger variable with the name `varname`. If the dimension is specified, the debugger variable will be an array of the specified size and shape.

## defset

*(application targeted)*

Summary: Define process set command.
Usage: `defset <name> <process set expression>`
Example command: `defset new_set $a,$b{3}`

Arguments: `name` (new process set name), `process set expression` (process set expression list)

Description:
Create a new process set of application ranks that can be focused upon. The values of the `process set expression` must be valid invoked application names and valid ranks if a particular rank is targeted. A set cannot contain other process sets defined through defset.

## delete

*(application targeted)*

Summary: Delete breakpoint command.
Usage: `delete [<breakpoints>]`
Example command: `delete 1`
Shortcut: `d`

Arguments: `breakpoints` (list of breakpoint numbers)

Description:
Delete the list of breakpoints specified by the breakpoints argument in every application rank of the current process set. If no argument is supplied, all breakpoints in the current process set are deleted.

## down

*(application targeted)*

Summary: Move stack frame down command.
Usage: `down [<num>]`
Example command: `down`

Arguments: `num` (number of frames to go down)

Description:
Move the call stack down by `num` frames. If no argument is supplied, then move down by one frame.

## focus

*(application independent)*

Summary: Focus process set command.
Usage: `focus <proc_set>`
Example command: `focus all`

Arguments: `proc_set` (named process set)

Description:
Change the focus of the current process set to the named process set specified by the argument `proc_set`.

## graph

*(application independent)*

Summary: Dataflow graph creation command.
Usage: `graph <name>`
Example command: `graph $my_graph`

Deferred mode commands: `assert, assign, end, help, set`
Arguments: `name` (debugger convenience variable)

Description:
Begin creation of a new dataflow graph. Graph sub-commands are read until `end` is encountered. See `usage assert` for more information.

## halt

*(application targeted)*

Summary: Halt all executing processes.
Usage: `halt`
Example command: `halt`

Arguments: None.

Description:
Halt all executing processes in the current process set. This is equivalent to sending a SIGINT to the processes.

## help

*(application independent)*

Summary: Help command.
Usage: `help [<command>]`
Example command: `help`
Shortcut: `h`

Arguments: `command` (command specific message)

Description:
Display help information about commands. If no argument is given, display a list of all valid commands.

## info

*(application independent)*

Summary: Info command.
Usage: `info [<subcommand>]`
Example command: `info`

Arguments: `subcommand` (info subcommand)

Description:
Display information relating to the application being debugged. If no argument is given, display a list of all valid info subcommands.

## invoke

*(application independent)*

Summary: Specify an application to be debugged.
Usage: `invoke <proc_list> <application> [--args <args>]`
`[--aprun-args <aprun_args>] [--aprun-input <path_to_file>]`
Example command: `invoke $a ./helloworld`

Arguments: `proc_list` (debugger convenience variable syntax), `application` (name of application), `args` (application arguments), `aprun_args` (aprun arguments), `path_to_file` (aprun stdin file)

Description:
Start an application with process set name of `proc_list` and optional application arguments `args`, aprun arguments `aprun_args`, and aprun input file `path_to_file`.

For sequential programs, `proc_list` is a single scalar convenience variable. For parallel programs, `proc_list` must be a list of one or more convenience variable arrays or multiple scalar convenience variables. Application ranks are assigned to each convenience variable in the `proc_list` sequentially, starting from the left.

The summation of all convenience variables, including the length of any array, is used to determine the number of application ranks that will be launched.

For example:
A serial application with a `proc_list` of `$a,$b,$c` would start 3 distinct application instances. A parallel application with a `proc_list` of `$a{9}` would start a single distinct application instance containing 9 application ranks. A parallel application with a `proc_list` of `$a,$b{8}` would start a single distinct application instance with 9 ranks that are separated into two process sets. Process set `$a` would contain one rank and process set `$b` would contain eight ranks.

The optional `args`, `aprun_args`, and `path_to_file` should be enclosed with quotes. The `args` argument is used to pass arguments directly to the application. The `aprun_args` argument is used to pass additional arguments to aprun when launching the application. The `path_to_file` argument is used to redirect the stdin of aprun to a file. This is useful when your application requires input from stdin in order to set itself up.

For example, to provide the aprun arguments `-N1 -L134` you would use:
`--aprun-args "-N1 -L134"`

## kill

*(application targeted)*

Summary: Kill invoked applications.
Usage: `kill`
Example command: `kill`

Arguments: None.

Description:
Kill all processes contained in the current process set.

## list

*(application targeted)*

Summary: List specified function or line command.
Usage: `list [-] [<expr>]`
Example command: `list 20`
Shortcut: `l`

Arguments: `-` (backward direction), `expr` (list expression syntax)

*List expression syntax*
The list expression can contain an argument of the following forms:

1. `linenum` - display source lines around specified line number.
2. `file:linenum` - display source lines around specified line number in specified source file.
3. `function` - display source lines around specified function.
4. `file:function` - display source lines around specified function in specified source file.
5. `*addr` - display source lines around the line corresponding to address `addr`.

Additionally, a list of two arguments can be defined in `expr` (separated with `,`) to display all source lines between arg1 and arg2.

Description:
Display source code of the processes in the current process set. With no argument, ten lines of code are displayed from the previous listing. If `-` is specified, ten previous lines of code are displayed.

## maint

*(application independent)*

Summary: Maint command.
Usage: `maint [<subcommand>]`
Example command: `maint`

Arguments: `subcommand` (maint subcommand)

Description:
Commands that are useful only to ccdb maintainers or are experimental. If no argument is given, display a list of all valid maint subcommands.

## next

*(application targeted)*

Summary: Next command.
Usage: `next [<num>]`
Example command: `next`
Shortcut: `n`

Arguments: `num` (number of lines to advance)

Description:
Step program, but do not follow into subroutine calls. If the argument `num` is supplied, advance `num` lines.

## print

*(application optional)*

Summary: Print the value of an expression.
Usage: `print <expr>`
Example command: `print prog_variable`
Shortcut: `p`

Arguments: `expr` (expression to print)

Description:
Print the value of `expr` for each application rank in the current process set. The `expr` can contain any combination of debugger convenience or application variables. Valid application variables are those of the lexical environment of each ranks selected stack frame, plus all those whose scope is global or over an entire file.

## quit

*(application independent)*

Summary: Quit command.
Usage: `quit`
Example command: `quit`
Shortcut: `q`

Arguments: None.

Description:
Quit the debugger.

## release

*(application targeted)*

Summary: Release debugged applications from debugger control.
Usage: `release <process_set>`
Example command: `release $a`

Arguments: `process_set` (named process set)

Description:
Release debugged applications specified by `process_set` from debugger control.

## set

*(application independent)*

Summary: Set command.
Usage: `set [<subcommand>]`
Example command: `set`

Arguments: `subcommand` (set subcommand)

Description:
Set information relating to ccdb environment settings and behavior. If no argument is given, display a list of all valid set subcommands.

## show

*(application independent)*

Summary: Show command.
Usage: `show [<subcommand>]`
Example command: `show`

Arguments: `subcommand` (show subcommand)

Description:
Show information relating to ccdb environment settings and behavior. If no argument is given, display a list of all valid show subcommands.

## source

*(application independent)*

Summary: Read and execute debugger commands.
Usage: `source <filename>`
Example command: `source myfile`

Arguments: `filename` (path to named command file)

Description:
Read debugger commands from a file to automate debugging tasks.

## start

*(application independent)*

Summary: Dataflow graph execution command.
Usage: `start <name>`
Example command: `start $my_graph`

Arguments: `name` (defined graph name)

Description:
Begin execution of a defined dataflow graph. Graph execution cycles until the dataflow interpreter halts due to encountering assertion failures or program errors.

## step

*(application targeted)*

Summary: Step command.
Usage: `step [<num>]`
Example command: `step`
Shortcut: `s`

Arguments: `num` (number of lines to step)

Description:
Advance the program to the next source line, following into subroutine calls. If the argument `num` is supplied, then advance `num` lines.

## stop

*(application independent)*

Summary: Dataflow graph halt command.
Usage: `stop`
Example command: `stop`

Arguments: None.

Description:
Inform the dataflow interpreter to halt the current graph execution immediately. This command is used to abort graph execution.

## up

*(application targeted)*

Summary: Move stack frame up command.
Usage: `up [<num>]`
Example command: `up`

Arguments: `num` (number of frames to go up)

Description:
Move the call stack up by `num` frames. If no argument is supplied, then move up by one frame.

## usage

*(application independent)*

Summary: Usage command.
Usage: `usage [<subcommand>]`
Example command: `usage`

Arguments: `subcommand` (usage subcommand)

Description:
Display information relating to deferred mode commands and their use. If no argument is given, display a list of all valid usage subcommands.

## viewset

*(application independent)*

Summary: View process set command.
Usage: `viewset [<proc_set>]`
Example command: `viewset`

Arguments: `proc_set` (named process set)

Description:
Display information about defined process sets. With no argument provided, information about all defined process sets is displayed.

## whatis

*(application targeted)*

Summary: Whatis data type of expression.
Usage: `whatis <expr>`
Example command: `whatis prog_var`

Arguments: `expr` (expression syntax)

*Expression syntax*
The `expr` argument can be the name of a program variable in an invoked application or an internal debugger convenience variable such as `$a`.

Description:
Display the data type of an expression for each application rank in the current process set.

## Info Subcommands

### info breakpoints

Summary: Breakpoint information.

Description:
Display the status of user defined breakpoints.
The `type` column indicates one of the following:
`breakpoint` normal breakpoint
`watchpoint` watchpoint
The `disp` column contains `keep`, `del`, or `dis` to indicate the disposition of the breakpoint after it is hit. If `disp` is displayed as `dis`, this means that the breakpoint will be disabled. The `address` and `what` columns indicate the address and file/line number of the breakpoint respectively.

### info blockmap

Summary: Display defined blockmaps.

Description:
Display the names of defined blockmaps.

### info diff

Summary: Saved graph diff information.

Description:
Display the set of saved difference data from graph assertions collected during graph execution. A numeric argument can be provided to view the actual deltas of the data type comparison.

### info graph

Summary: Display defined dataflow graphs.

Description:
Display the names of defined dataflow graphs. A name argument can be provided to view the assertions defined in the named graph.

### info gstatus

Summary: Graph status information.

Description:
Display summary information relating to actively executing graphs.

### info locals

Summary: Local symbol information.

Description:
Display a list of the local symbols in the current stack frame for each application rank in the current process set.

### info process

Summary: Process information.

Description:
Display information related to currently invoked processes. The `name` column specifies the process set name used to identify the invoked application. The `type` column indicates if the process is serial or parallel. The `procs` column indicates the number of application ranks. The `focus` column indicates if this applications process set is currently focused upon. The `command` column identifies the command that was passed to aprun in order to launch the application.

### info status

Summary: Status command.

Description:
Display the status of invoked applications.

## Set Subcommands

### set async

Summary: Async mode.
Usage: `set async <on|off>`
Example command: `set async on`

Default: `on`
Arguments: `on` or `off`

Description:
Turn asynchronous mode `on` or `off`. This determines if the debugger will accept additional commands before the current command completes.

### set dfmode

Summary: Dataflow processing mode.
Usage: `set dfmode <default|sync|async>`
Example command: `set dfmode default`

Scope: Graph
Default setting: `default`
Arguments: `default` or `sync` or `async`

Description:
Set the dataflow processing mode to synchronous or asynchronous. This changes the dataflow graph that was generated using the `graph` command so that it operates in one of these three modes. Synchronous mode is default for assertions that refer to independent processes. Asynchronous mode is default for assertions that refer to the same process. Using this setting overrides the default behavior.

## set directory

Summary: Directory search source path.
Usage: `set directory <path>`
Example command: `set directory /path/to/source`

Arguments: `path` (directory containing source)

Description:
Add the directory defined by `path` to the beginning of the search path for the application source files that were used to generate symbol information for application. Typically this is used when an application was compiled on a different machine than run, but the source files used to compile the application are available at a new location. This will cause cached information about source file locations and line positions to be lost.

## set display

Summary: Dataflow graph assert display mode.
Usage: `set display <text|aif|hdf|hdf5>`
Example command: `set display text`

Scope: Graph
Default setting: `text`
Arguments: `text` or `aif` or `hdf` or `hdf5`

Description:
Set the display format for the result of dataflow graph assertions. In `text` mode, any differences are displayed as plaintext. In `aif` mode, differences are displayed in architecture independent format (AIF). In `hdf` and `hdf5` modes, differences are displayed in the hierarchical data format (HDF) version 4 and version 5 respectively.

## set error

Summary: Dataflow graph assert lower and upper floating point error limits.
Usage: `set error <lower limit> <upper limit> [relative|absolute]`
Example command: `set error 0.5 0.5 absolute`

Scope: Graph
Default setting: `ulp ulp absolute`
Where `ulp` is the smallest non-zero floating point value of the current machine.
Arguments: `lower limit` (float), `upper limit` (float), `relative` or `absolute`

Description:
Set the tolerance level for the dataflow graph assertion difference deltas when comparing floating point values. Differences with deltas below the `lower limit` are ignored. Differences with deltas between the `lower limit` and the `upper limit` will be displayed using the current display mode setting. Any differences above the `upper limit` will halt the dataflow interpreter and stop graph execution. The `relative` and `absolute` limits are used to specify how the lower and upper limits should be interpreted. If `relative` is selected, the difference deltas between values in an assertion is divided by the larger of the original two values used to calculate the delta.

## set force

Summary: Dataflow graph assert force mode.
Usage: `set force [on|off]`
Example command: `set force off`

Scope: Graph
Default setting: `off`
Arguments: `on` or `off`

Description:
Determines whether subsequent dataflow graph assertions will copy the value of the left-variable (LHS) into the right-variable (RHS) after the assertion has finished executing.

## set function

Summary: Set function definition command.
Usage: `set function <function_name>(<function_args>) = <expr>`
Example command: `set function myfunc(myvar) = 1`

Arguments: `function_name` (name of function), `function_args` (function arguments), `expr` (function expression definition)

Description:
Define a function macro named `function_name`. The expression `expr` can contain format arguments that are substituted by actual arguments supplied to the function. Recursive function expression calls are not allowed.

## set halt

Summary: Dataflow interpreter halting mode.
Usage: `set halt [true|false]`
Example command: `set halt true`

Scope: Graph
Default setting: `true`
Arguments: `true` or `false`

Description:
Determines if application ranks are halted when the dataflow interpreter terminates.

## set logfile

Summary: Dataflow event logfile name.
Usage: `set logfile <filename>`
Example command: `set logfile "my.file"`

Scope: Graph
Default setting: `null`
Arguments: `filename` (quoted string)

Description:
Specifies the name of the file to send dataflow event logging entries if dataflow event logging is turned on. Note that the provided filename must be enclosed by quotes.

## set logging

Summary: Dataflow event logging mode.
Usage: `set logging [on|off]`
Example command: `set logging on`

Scope: Graph
Default setting: `off`
Arguments: `on` or `off`

Description:
Turn dataflow event logging on or off.

## set output

Summary: dataflow graph assertion result logfile name.
Usage: `set output <filename>`
Example command: `set output "my.file"`

Scope: Graph
Default setting: `null`
Arguments: `filename` (quoted string)

Description:
Specify a file to send the results from dataflow graph assertions to. Note that only difference delta information from the execution of an assertion will be written. The provided filename must be enclosed by quotes.

## set reduce

Summary: Dataflow reduce mode.
Usage: `set reduce [sum|product|mean|aphash|fnvhash|bobhash|none] [<lower precision>] [<upper precision>]`
Example command: `set reduce aphash`

Scope: Graph
Arguments: `sum` or `product` or `mean` or `aphash` or `fnvhash` or `bobhash` or `none`, `lower precision` (float), `upper precision` (float)

Description:
Use a data reduction function to reduce the volume of data sent from compute nodes. There are both simple and complex functions currently supported for data reduction. The simple functions are sum, product, and mean which compute the sum, product, or mean of the data structure in question.

Currently there are three hash functions supported which will produce more reliable results. Jenkins hash function (bobhash), Fowler–Noll–Vo hash function (fnvhash), and Arash Partow hash function (aphash). Note that these hashing functions are for data reduction purposes only and are not cryptographically secure. When hashing is used, the two original sets of data will no longer be directly compared. Instead, two sets of hashed signatures will be compared. If a blockmap is used to compare a serial data structure to an n-ary distributed data structure, then the serial program will calculate n hashes and compare it with its associated counterpart in the n-ary distributed data structure.

If floating point values are being used with a hashing function, the user can specify the precision tolerance (lower and upper) for the hash function using the optional `lower precision` and `upper precision` arguments. For example, issuing the command `set reduce bobhash 1 2` will set the reduce function to Jenkins hash with floating point lower precision and upper precision of 10^-1 and 10^-2 respectively.

## Show Subcommands

### show async

Summary: Async setting.

Description:
Display the current async setting.

### show dfmode

Summary: Dataflow graph processing setting.

Description:
Display the current dataflow graph processing mode setting. When set to default, synchronous mode is used for assertions that refer to independent processes. Asynchronous mode is default for assertions that refer to the same process. The value of this setting overrides the default behavior.

### show directories

Summary: Display source search path.

Description:
Display the current search path for locating source files. A `$cwd` in the path means the current working directory. A `$cdir` in the path means the compilation directory of the source file.

### show display

Summary: Dataflow graph assert display setting.

Description:
Display the current display format for the result of dataflow graph assertions. In `text` mode, any differences are displayed as plaintext. In `aif` mode, differences are displayed in architecture

independent format (AIF). In `hdf` and `hdf5` modes, differences are displayed in the hierarchical data format (HDF) version 4 and version 5 respectively.

## show error

Summary: Dataflow graph assert lower and upper floating point error limits.

Description:
Display the current tolerance level setting for the dataflow graph assertion difference deltas. Differences with deltas below the `lower limit` are ignored. Differences with deltas between the `lower limit` and the `upper limit` will be displayed using the current display mode setting. Any differences above the `upper limit` will halt the dataflow interpreter and stop graph execution. The `relative` and `absolute` limits are used to specify how the lower and upper limits should be interpreted. If the setting is `relative`, the difference deltas between values in an assertion is divided by the larger of the original two values used to calculate the delta.

## show force

Summary: Assert command force mode.

Description:
Display the current setting of the force mode for dataflow graph assertions. Determines whether subsequent dataflow graph assertions will copy the value of the left-variable (LHS) into the right-variable (RHS) after the assertion has finished executing.

## show function

Summary: Function definition information.

Description:
Display a list of the currently defined function macros.

## show halt

Summary: Dataflow interpreter halting mode.

Description:
Display the current setting of the dataflow interpreter halting mode. Determines if application ranks are halted when the dataflow interpreter terminates.

## show logfile

Summary: Dataflow event logfile name.

Description:
Display the current name of the file to send dataflow event logging entries if dataflow event logging is turned on.

## show logging

Summary: Dataflow event logging mode.

Description:
Display the current mode of dataflow event logging.

### show output

Summary: dataflow graph assertion result logfile name.

Description:
Display the file to send the results from dataflow graph assertions to. Only difference delta information from the execution of an assertion will be written to this file.

### show reduce

Summary: Dataflow reduce mode.

Description:
Display the current setting of the dataflow reduce mode. There are both simple and complex functions currently supported for data reduction. The simple functions are sum, product, and mean which compute the sum, product, or mean of the data structure in question.

Currently there are three hash functions supported which will produce more reliable results. Jenkins hash function (bobhash), Fowler–Noll–Vo hash function (fnvhash), and Arash Partow hash function (aphash). Note that these hashing functions are for data reduction purposes only and are not cryptographically secure. When a hashing scheme is used, the two original sets of data will no longer be directly compared. Instead, two sets of hashed signatures will be compared. If a blockmap is used to compare a serial data structure to an n-ary distributed data structure, then the serial program will calculate n hashes and compare it with its associated counterpart in the n-ary distributed data structure.

If floating point values are being used with a hashing function, the precision tolerance (lower and upper) for the hash function is determined by the `lower precision` and `upper precision` settings.

### show version

Summary: Version information.

Description:
Display the debugger version number.

## Usage Subcommands

### usage assert

Summary: Dataflow graph assert command.

Description:
Display help information regarding the use of dataflow graph `assert` commands. The `assert` command is only valid while defining a graph in deferred mode.

### usage assign

Summary: Dataflow graph assign command.

Description:
Display help information regarding the use of dataflow graph `assign` commands. The `assign` command is only valid while defining a graph in deferred mode.

## usage define

Summary: Blockmap define command.

Description:
Display help information regarding the use of blockmap `define` commands. The `define` command is only valid while defining a blockmap in deferred mode.

## Blockmap Deferred Mode Commands

## define

Summary: Blockmap define command.
Usage: `define [data|distribute|grid|dim_order](<dimensional definition>)`
Example command: `define distribute(block,*)`

Arguments: `data` or `distribute` or `grid` or `dim_order` , `dimensional definition` (dimension/rank/distribution information)

Description:
Define a blockmap `data`, `distribute`, `grid`, or `dim_order` definition. The `data` definition is used to describe the size and dimensionality (or rank) of the global reconstruction. The `distribute` definition is used to describe the distribution for each dimension of the reconstruction. The `grid` definition is used to describe the processing grid (or number of ranks contained in each dimension) for the reconstruction. The `dim_order` definition is used to describe the fastest varying dimension of the global reconstruction for rank assignment (i.e. which dimension contains sequential numbered ranks; and the order in which the other dimensions are incremented). The dimensionality of all provided definitions must match. Thus if the user supplied a `data` definition of `data(32,64,1024)` they must also provide a three dimensional `distribute`, `grid`, and `dim_order` definition describing the behavior for each of the three dimensions.
The `data` definition accepts a numerical list with each entry separated by a comma to define the global data size in each dimension for the reconstruction. For example, a `data` definition of `data(32,64,1024)` would define a three dimensional global array of size 32x64x1024. The first entry in the list represents the first dimension of the global reconstruction, the second entry represents the second dimension, and so on.

The `grid` definition accepts a numerical (or '*', more on this below) list with each entry separated by a comma to define the number of ranks distributed in each dimension of the global reconstruction. For example, a `grid` definition of `grid(2,4,64)` would define a processing grid of 512 ranks (2*4*64=512) with two ranks distributed in the first dimension, four ranks distributed in the second dimension, and sixty four ranks distributed in the third dimension. As with the `data` definition, the first entry in the list represents the first dimension in the processing grid of the global reconstruction and so on.

The `distribute` definition accepts a list of either `block`, `cyclic`, a numerical value, or * which is again separated by a comma to define the distribution in each dimension of the global reconstruction. The `block` keyword is used when equal sized chunks of data are assigned to each rank. The size of the chunk is determined by dividing the `data` size for the dimension divided by `grid` size for the dimension. Uneven chunks are not currently supported. The `cyclic` keyword is used when elements in the dimension are dealt out in round robin fashion. The number of ranks that are dealt over is determined by the `grid` size for that dimension. A numerical value is used to describe a block-cyclic

distribution scheme. The numerical value represents the blocking factor used to partition the global reconstruction's dimension into blocks that the processing grid is mapped onto. The '*' value represents that the distribution scheme for that dimension is ignored. Each rank in the global reconstruction would contain all of the data in that dimension. This can be thought of as columns (or rows) of data for that particular dimension. For '*' distributed dimensions, the '*' keyword should also be provided to the `grid` and `dim_order` definitions for that dimension since there is no distribution present and thus no ranks are assigned to that particular dimension (every rank contains that dimensions data).

The `dim_order` definition accepts a numerical (or '*', see above) list with each entry separated by a comma to define the order in which application ranks are assigned in each dimension of the process grid for the global reconstruction. Each local "chunk" of data obtained from each rank needs to be placed into the global reconstruction. In order to do this, each rank needs to be assigned a logical position in the processing grid for its "chunk" of data. When dealing with n-dimensional distributions, any of the n dimensions could be assigned sequential numbered ranks and any of the other higher order dimensions could be incremented after the dimension containing sequential ranks is filled. To define this order, each of the distributed dimensions is assigned a sequential number, starting from 1 up to n, to describe the fastest-to-slowest varying dimensions. The fastest varying dimension is the dimension that is assigned sequential ranks up to its corresponding grid size. The second fastest varying dimension is incremented after the fastest varying dimension is completely filled and ranks are again assigned to the fastest varying dimension. This process continues until all ranks have been assigned to all of the n-dimensions. Again, any dimension that is not distributed should receive a '*' for its corresponding `dim_order` dimension since all of the data in that dimension is present on every rank.

## end

Summary: Blockmap end command.
Usage: `end`
Example command: `end`

Arguments: None.

Description:
This command is used to end the current blockmap definition. After issuing `end`, the debugger will exit deferred mode and attempt to create a blockmap function for future use. The user will be warned if there were any errors that prevented the blockmap function from being created.

## help

Summary: Blockmap help command.
Usage: `help [<command>]`
Example command: `help`
Shortcut: `h`

Arguments: `command` (command specific message)

Description:
Display help information about blockmap deferred mode commands. If no argument is given, display a list of all valid blockmap deferred mode commands commands.

## usage

Summary: Usage command.
Usage: `usage [<subcommand>]`
Example command: `usage`

Arguments: `subcommand` (usage subcommand)

Description:
Display information relating to deferred mode commands and their use. If no argument is given, display a list of all valid usage subcommands.

## Graph Deferred Mode Commands

### assert

Summary: Dataflow graph assert command.
Usage: `assert <expr1> = <expr2>`
Example command:
```
assert $app1::var1@"app1.c":10 = $app2::var2@"app2.c":20
```

Arguments: `expr1` (assert expr syntax), `expr2` (assert expr syntax)

Assert expression syntax:
Expr Usage: `<procset>::<var>@"<source file>":<line>`
Expr arguments: `procset` (defined process set name), `var` (application variable), `source file` (quoted source file name), `line` (`line` in source file to reference `var`)
Expr example: `$my_app::my_var@"my_source.c":10`

Description:
Assert that the variable `var` defined in `expr1` is equivalent to the variable `var` defined in `expr2`. Variables are specified using the *assert expression syntax* which declares both spatial and temporal references for a variable.

Define the spatial locality with `$procset::var` to specify the particular variable of interest within a set of application ranks. This is followed with the temporal reference of `"source file":line` to define the point within application execution where to examine the value of the variable. A blockmap can be used in the expr by enclosing the expr with curly braces and providing the name of the blockmap in front. For example, to use the blockmap named `array_a` on the left hand of the expression and the blockmap named `array_b` on the right hand, provide the assertion:

```
assert array_a{expr1} = array_b{expr2}
```

The result of the assertion command depends on the current settings of `error`, `display`, `output`, and `logging`.

### assign

Summary: Dataflow variable assignment command.
Usage: `assign <convenience var> <application var> [-force]`
Example command: `assign $saved_var $procset::var`

Arguments: `convenience var` (debugger convenience variable), `application var` (application variable), `-force` (optional force mode)

Description:
Assign the value of the application variable `application var` to internal debugger convenience variable `convenience var`. The saved application variable will then be available for use in later expressions during application execution.

## end

Summary: Dataflow graph end command.
Usage: `end`
Example command: `end`

Arguments: None.

Description:
This command is used to end the current graph definition. After issuing `end`, the debugger will exit deferred mode and attempt to compile the defined graph with the dataflow compiler. The user will be warned if there were any errors that prevented the graph from being compiled. Upon successful compilation, the graph can be run by the dataflow interpreter by issuing the `start` command.

## help

Summary: Help command.

Description:
Display help information about graph deferred mode commands.

## set

Summary: Set command.
Usage: `set [<subcommand>]`
Example command: `set`

Arguments: `subcommand` (set subcommand)

Description:
Set information relevant to the scope of the dataflow graph relating to ccdb environment settings and behavior. If no argument is given, display a list of all valid set subcommands.

## show

Summary: Show command.
Usage: `show [<subcommand>]`
Example command: `show`

Arguments: `subcommand` (show subcommand)

Description:
Show information relevant to the scope of the dataflow graph relating to ccdb environment settings and behavior. If no argument is given, display a list of all valid show subcommands.

## usage

Summary: Usage command.

*Usage*: `usage [<subcommand>]`
*Example command*: `usage`

*Arguments*: `subcommand` *(usage subcommand)*

*Description*:
*Display information relating to deferred mode commands and their use. If no argument is given, display a list of all valid usage subcommands.*

© 2011-2012 Cray Inc. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior permission of the copyright owners.

Cray is a registered trademark, and the Cray logo, Cray XE6, Cray XT6, Cray XT5, Cray XT, Cray XT4 and Cray XT3 are trademarks of Cray Inc. Other product and service names mentioned herein are the trademarks of their respective owners